



Universidad Nacional Autónoma de México.

FACULTAD DE INGENIERÍA.

Proyecto Final

Bitácora IA

Estructura y Programación de Computadoras

Profesora: Ing. Adara Mercado Martínez

-Integrantes:

- Carranza Gonzáles Alexis Aaron
- Martínez Márquez Azael
- Navarro Saldaña Ivan
- Perez Gonzáles Natalia

Fecha de entrega: 11 de junio de 2026

1. Herramientas de Inteligencia Artificial Utilizadas

Herramienta	Integrante(s)	Uso
Claude (Anthropic)	Azael, Iván, Aaron, Natalia	Ensamblador de 2 pasadas, linker, depuración de errores, organización del proyecto
Gemini (Google)	Azael, Iván, Aaron, Natalia	Consultas sobre arquitectura IA-32, opcodes, verificación de conceptos y ensamblador de 1 pasada

Se utilizó Gemini (Google) como asistente de programación y depuración. La herramienta se empleó principalmente para analizar errores de compilación en C, estructurar archivos de cabecera (.h), depurar errores de enlazador (*linker*) y generar expresiones regulares/lógica de traducción de ensamblador a código máquina.

Se utilizó Claude (Anthropic) como asistente principal de desarrollo y depuración para el ensamblador de 2 pasadas. La herramienta se empleó a lo largo de todo el proyecto: desde la generación del código base de cada módulo (lexer, parser, encoder, symtable, objwriter y linker), hasta la identificación y corrección de bugs encontrados durante la ejecución real del ensamblador. Claude fue especialmente útil para depurar errores de lógica difíciles de detectar, como el manejo incorrecto de “**current_section**” entre módulos. También se utilizó para organizar la integración de ambos proyectos en un único repositorio y para generar los casos de prueba, el Makefile y el script de automatización.

2. Prompts Utilizados

2.1 Caso Gemini

La comunicación se basó en iteraciones directas sobre los errores de la terminal y la estructura del proyecto. Los prompts más relevantes fueron:

- *"Necesito que arregles estos errores de mi código teniendo en cuenta la estructura de los archivos..."* (Para el diagnóstico inicial de la arquitectura).
- *"Te paso los archivos que tienen [mis compañeros] para que nos acomodemos"* (Para integrar el código del equipo trabajado en Live Share).
- *"Me marca undefined reference to encode_instruction... y implicit declaration of function..."* (Para resolver problemas críticos de enlazado y visibilidad de

funciones).

- *"Pásame el código completo ya corregido"* (Para consolidar las soluciones fragmentadas en archivos completos y evitar errores de sintaxis).
- *"make: *** No rule to make target 'obj/main_1pass.o'..."* (Para corregir conflictos en el sistema de construcción automatizada).

2.2 Caso Claude

2.2.1. Generación de código base

- Solicitar la estructura inicial del lexer, parser, symtable y encoder para un ensamblador IA-32 en C.

Prompt utilizado: *"Actúa como un experto en compiladores en C. Necesito la estructura base (archivos .h y .c) para un lexer, parser, tabla de símbolos y encoder de un ensamblador de dos pasadas para la arquitectura IA-32."*

- Pedir la implementación del formato objeto UNAM con header, tabla de símbolos y tabla de relocalaciones.

Prompt utilizado: *"Escribe una función en C que genere un formato objeto binario personalizado. Debe escribir un ObjHeader con la firma 'UNAM', seguido del arreglo de estructuras ObjSymbol y finalmente la tabla de ObjRelocation."*

- Solicitar el mini linker capaz de leer múltiples archivos .o, resolver relocalaciones y generar un binario final.

Prompt utilizado: *"Necesito implementar un mini linker en C. Debe abrir dos archivos .o en formato binario, leer sus encabezados para sumar los tamaños de las secciones .text y .data, y parchar las direcciones pendientes usando las tablas de relocalaciones para generar un ejecutable final en binario."*

2.2.2 Depuración y corrección de errores

- *"Tengo estos errores de compilación [output de make], ayúdame a corregirlos."*
- *"El ensamblador genera 0 bytes de código máquina, encuentra el bug."*
- *"current_section siempre vale 0 aunque el parser procesa SECTION .text, encuentra por qué."*
- *"Los flags is_global e is_extern están incorrectos en el archivo objeto, corrígelos."*
- *"RESB, RESW, RESD no incrementan el contador de BSS, encuentra el"*

problema."

2.3 Mejoras e implementación incremental

- Solicitar soporte para directivas DB, DW, DD con strings y números.

Prompt utilizado: *"Agrega soporte al parser y encoder para las directivas DB, DW y DD. El lexer ya reconoce cadenas, pero necesito que procese correctamente la reserva de memoria con valores múltiples, como mensaje DB \"Hola\", 0x0A."*

- Pedir implementación de ORG y EQU para constantes simbólicas.

Prompt utilizado: *"¿Cómo implemento la directiva ORG para modificar el contador de programa local y la directiva EQU para guardar constantes inmediatas en mi tabla de símbolos durante la primera pasada?"*

- Solicitar memoria directa [dirección] en el encoder.

Prompt utilizado: *"¿Cómo implemento la directiva ORG para modificar el contador de programa local y la directiva EQU para guardar constantes inmediatas en mi tabla de símbolos durante la primera pasada?"*

- Pedir los saltos condicionales faltantes: JNE, JG, JL, JGE, JLE.

Prompt utilizado: *"Ayúdame con los opcodes exactos en IA-32 para los saltos condicionales JNE, JG, JL, JGE y JLE, y con cálculo del offset relativo en complemento a dos hacia atrás y hacia adelante?"*

2.4 Organización del proyecto

- Solicitar estructura de carpetas para integrar ensamblador de 1 y 2 pasadas en un solo repositorio.

Prompt utilizado: *"Tengo el código de un ensamblador de 1 pasada y otro de 2 pasadas. Dame una estructura de carpetas estándar (src/, include/, tests/) donde ambos proyectos puedan existir sin sobrescribirse."*

- Pedir Makefile que compile ambos proyectos de forma independiente.

Prompt utilizado: *"Escribe un Makefile que use la función filter-out para ignorar archivos main específicos y así poder compilar ./assembler_1pass y*

./assembler_2pass de manera simultánea compartiendo el mismo núcleo del código (lexer, parser, etc.)."

- Solicitar script run_tests.sh que pruebe ambos ensambladores y el linker.

Prompt utilizado: *"Escribe un script llamado run_tests.sh que ensamble todos los archivos de prueba en la carpeta tests/ y ejecute el linker automáticamente."*

3. Qué código fue generado automáticamente

La IA generó y/o reestructuró el código de los siguientes módulos clave a lo largo del proyecto:

Para el ensamblador de 1 pasada, Gemini generó aproximadamente 700 líneas de código en C:

- **encoder.h y encoder.c:** Prototipos y lógica completa para la generación de opcodes, ModRM y SIB, así como el manejo del buffer de memoria (machine_code_buffer).
- **main_1pass.c:** Orquestador principal adaptado para hacer una sola lectura del código fuente, integrando la resolución de saltos adelantados.
- **objfile.c:** Estructuras y lógica binaria para generar el archivo objeto final con el encabezado "UNAM" y la tabla de símbolos.
- **Makefile:** Reglas de compilación automatizadas para generar múltiples ejecutables (2 pasadas vs 1 pasada) sin conflictos de funciones main.

Para el ensamblador de 2 pasadas, Claude generó los siguiente:

- **lexer.c**
- **parser.c**
- **encoder.c**
- **symtable.c**
- **objwriter.c**
- **linker.c**

- **main_2pass.c**

Esto incluye el tokenizador completo con soporte para strings y números hexadecimales, el parser de instrucciones con todos los modos de direccionamiento IA-32, la generación de opcodes con ModRM y SIB, la tabla de símbolos con soporte para `is_global`, `is_extern` y `section_id`, la escritura del archivo objeto en formato UNAM, el mini linker con resolución de relocalaciones y fusión de secciones, y el Makefile final con compilación independiente de ambas pasadas.

4. Qué modificaciones realizaron manualmente

Aunque el código base fue generado por IA, el equipo realizó las siguientes modificaciones y contribuciones manuales durante el desarrollo:

4.1 Corrección de bugs encontrados en ejecución

- Corrección del bug donde `current_section` siempre valía 0 al inicio del PASS 2, causando que todos los bytes se emitieran a la sección incorrecta. Causa: el parser comparaba `'.text'` en minúsculas pero el lexer convertía todo a mayúsculas.
- Corrección de firma incompatible entre `symtable.h` y `symtable.c` para la función `add_symbol`, que causaba error de compilación por tipos conflictivos.
- Corrección del orden de parámetros en las llamadas a `symtable_add` después de agregar el parámetro `is_extern`, que causaba que todos los símbolos quedaran marcados como `extern=1`.
- Corrección del bloque de conteo de BSS que estaba anidado incorrectamente dentro del bloque de sección `.data`, causando que `lc_bss` siempre valiera 0.
- Corrección del nombre del archivo de salida hardcodeado como `'hello.o'` — se implementó derivación automática del nombre desde el archivo fuente.
- Ajuste manual de tamaños de buffer (unificando a 8192 bytes) para evitar colisiones de memoria identificadas durante el enlazado.

4.2 Integración de ambos proyectos

- Reorganización manual de la estructura de carpetas para separar src/1pass, src/2pass, include/1pass e include/2pass, separando los archivos de manera lógica para evitar colisiones.
- Actualización de rutas de #include en todos los archivos .c de ambas pasadas.
- Integración del modo --link dentro de main_2pass.c para que el mismo ejecutable funcione como ensamblador y como linker.
- Integración de la lógica del Lexer, Parser y Symtable trabajada en conjunto con los compañeros a través de VS Code Live Share.

4.3 Diseño y escritura de casos de prueba

- Diseño y escritura manual de test_basic.asm, test_jumps.asm, test_sib.asm.
- Diseño de test_linker_mod1.asm y test_linker_mod2.asm para probar resolución de símbolos externos entre módulos.
- Verificación manual de los bytes de código máquina generados contra la referencia IA-32.
- Ejecución constante de ciclos de limpieza y compilación (make clean y make) y validación de los archivos .o resultantes utilizando archivos de prueba .asm.

5. Qué errores encontraron

Durante el proceso de desarrollo, nos enfrentamos a varios errores técnicos principales que fueron resueltos poco a poco:

- **Undefined Reference (Error de Enlazador):** Ocurrió porque el archivo encoder.c no tenía el cuerpo de las funciones, dejando al enlazador sin instrucciones para compilar.
- **Implicit Declaration of Function:** Funciones como emit_byte o read_file eran llamadas antes de ser declaradas en sus respectivos archivos .h, lo que confundía al compilador.
- **Conflictos en el Makefile:** El script de compilación intentaba enlazar main.c y

main_1pass.c simultáneamente, causando un error de múltiples puntos de entrada.

- **Buffer Size Mismatch:** Diferentes archivos declaraban el buffer de código máquina con distintos tamaños (4096 vs 8192), lo que generaba riesgos de desbordamiento y errores de compilación.
- **Emisión de bytes en sección incorrecta y archivos .o vacíos:** Ocurrió porque el parser comparaba directivas como “.text” en minúsculas, mientras que el lexer convertía todo a mayúsculas, por lo que la variable “current_section” nunca cambiaba. Se resolvió modificando las comparaciones a ‘.TEXT’, ‘.DATA’ y ‘.BSS’ en mayúsculas directamente en el archivo parser.c.
- **Conflicting types for 'add_symbol':** Se debió a que la firma de la función en symtable.h utilizaba el tipo uint8_t, pero su implementación en symtable.c esperaba un int. La solución fue unificar la firma en ambos archivos para usar exactamente los mismos tipos de datos.
- **Banderas is_extern invertidas en símbolos:** Símbolos como “FUNCION_EXTERNA” se marcaban con “is_extern=0” y “_START” con “is_extern=1”. Esto sucedía porque el parámetro se calculaba ignorando el valor recibido y el orden de los parámetros en las llamadas estaba invertido. Se arregló utilizando el parámetro “is_extern” directamente y corrigiendo el orden en todas las llamadas a “symtable_add”.
- **Contadores de sección BSS estancados en 0:** Causado por un error lógico donde el bloque de conteo para directivas como “RESB”, “RESW” y “RESD” quedó anidado como un “else if” dentro del bloque de la sección “.data”. La solución fue mover este bloque al mismo nivel jerárquico independiente que “.text” y “.data”.
- **Cálculo de offset incorrecto en instrucciones CALL externas:** Al registrar “FUNCION_EXTERNA” con la propiedad “defined=0”, el encoder la encontraba y le asignaba una dirección local de “0x00” en vez de marcarla como una relocación pendiente. Se solucionó agregando una validación para verificar que

“sym->defined==1” antes de utilizar la dirección del símbolo.

- **Error fatal de compilación por rutas inexistentes:** Al reorganizar las carpetas para la integración de proyectos, las rutas relativas en “src/1pass/” apuntaban a directorios de cabeceras que ya no existían en esa ubicación. Se resolvió actualizando manualmente todos los “#include de ../include/” a la nueva ruta “../../include/1pass/”.